

Crossformer : Transformer utilizing cross-dimension dependency for multivariate time series forecasting

고려대학교 산업경영공학과
데이터 애널리틱스 및 헬스케어 시스템 연구실(DAHS)
석사과정
김홍범

1. Introduction

Published as a conference paper at ICLR 2023

CROSSFORMER: TRANSFORMER UTILIZING CROSS-DIMENSION DEPENDENCY FOR MULTIVARIATE TIME SERIES FORECASTING

Yunhao Zhang & Junchi Yan*

MoE Key Lab of Artificial Intelligence, Shanghai Jiao Tong University and Shanghai AI Lab
{zhangyunhao, yanjunchi}@sjtu.edu.cn
Code: <https://github.com/Thinklab-SJTU/Crossformer>

ABSTRACT

Recently many deep models have been proposed for multivariate time series (MTS) forecasting. In particular, Transformer-based models have shown great potential because they can capture long-term dependency. However, existing Transformer-based models mainly focus on modeling the temporal dependency (cross-time dependency) yet often omit the dependency among different variables (cross-dimension dependency), which is critical for MTS forecasting. To fill the gap, we propose Crossformer, a Transformer-based model utilizing cross-dimension dependency for MTS forecasting. In Crossformer, the input MTS is embedded into a 2D vector array through the Dimension-Segment-Wise (DSW) embedding to preserve time and dimension information. Then the Two-Stage Attention (TSA) layer is proposed to efficiently capture the cross-time and cross-dimension dependency. Utilizing DSW embedding and TSA layer, Crossformer establishes a Hierarchical Encoder-Decoder (HED) to use the information at different scales for the final forecasting. Extensive experimental results on six real-world datasets show the effectiveness of Crossformer against previous state-of-the-arts.

1 INTRODUCTION

Multivariate time series (MTS) are time series with multiple dimensions, where each dimension represents a specific univariate time series (e.g. a climate feature of weather). MTS forecasting aims to forecast the future value of MTS using their historical values. MTS forecasting benefits the decision-making of downstream tasks and is widely used in many fields including weather (Angryk et al., 2020), energy (Demirel et al., 2012), finance (Patton, 2013), etc. With the development of deep learning, many models have been proposed and achieved superior performances in MTS forecasting (Lea et al., 2017; Qin et al., 2017; Flunkert et al., 2017; Rangapuram et al., 2018; Li et al., 2019a; Wu et al., 2020; Li et al., 2021). Among them, the recent Transformer-based models (Li et al., 2019b; Zhou et al., 2021; Wu et al., 2021a; Liu et al., 2021a; Zhou et al., 2022; Chen et al., 2022) show great potential thanks to their ability to capture long-term temporal dependency (cross-time dependency).

Besides cross-time dependency, the cross-dimension dependency is also critical for MTS forecasting, i.e. for a specific dimension, information from associated series in other dimensions may improve prediction. For example, when predicting future temperature, not only the historical temperature, but also historical wind speed helps to forecast. Some previous neural models explicitly capture the cross-dimension dependency, i.e. preserving the information of dimensions in the latent feature space and using convolution neural network (CNN) (Lai et al., 2018) or graph neural network (GNN) (Wu et al., 2020; Cao et al., 2020) to capture their dependency. However, recent Transformer-based models only implicitly utilize this dependency by embedding. In general, Transformer-based models embed data points in all dimensions at the same time step into a feature vector and try to capture dependency among different time steps (like Fig. 1 (b)). In this way, cross-time dependency is well captured, but cross-dimension dependency is not, which may limit their forecasting capability.

*Junchi Yan is the correspondence author. This work was in part supported by NSFC (61972250, U19B2035, 62222607) and Shanghai Municipal Science and Technology Major Project (2021SHZDZX0102).

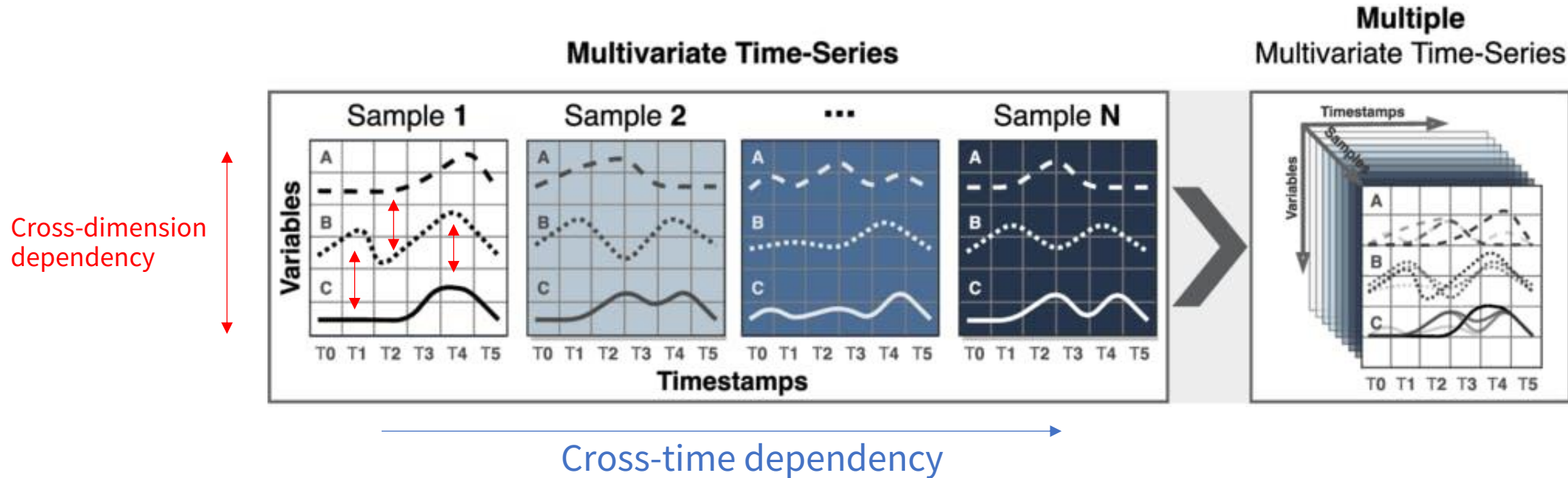
**“Crossformer: Transformer Utilizing Cross-Dimension Dependency for Multivariate Time Series Forecasting”
, 16 Feb 2023**

Y Zhang, J Yan

Citation : 330, ICLR 2023

2. Introduction

MTS(Multivariate time series) : aims to forecast the future value of MTS using their historical values.



RNN, LSTM, Transformer : Capture long-term temporal dependency (Cross-time dependency)

CNN(Convolution Neural Network), GNN(Graph Neural Network) : Preserving the information of dimensions in the latent feature space (Cross-dimension dependency)

3. Related work

Multivariate Time series Forecasting

- Statistical model : Vector auto-regressive(VAR), Vector auto-regressive moving average (VARMA) – Linear cross-dimension and cross-time dependency

Neural model

- TCN(CNN), DeepAR(RNN) – Cross-dimension,
- LSTnet(RNN – Cross-time, CNN -Cross-dimension)
- GNNs(Temporal convolution – Cross-time , Graph convolution - Cross-dimension)

Transformers for MTS forecasting

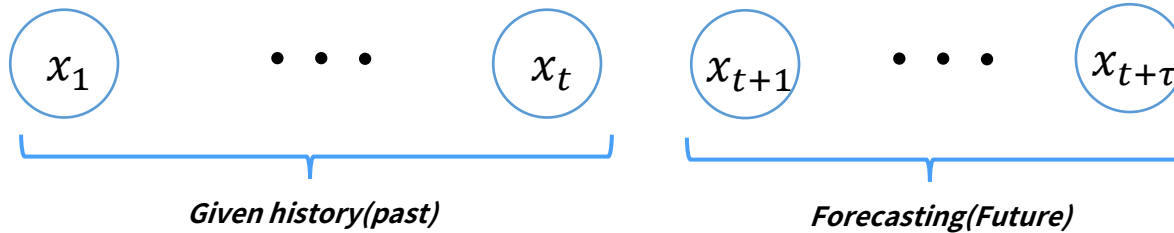
- LogTrans, Informer, Autoformer, Pyraformer, FEDformer, Preformer
- Focus on reducing the complexity of cross-time dependency(limitation)

Vision Transformer

- ViT – Split an image into non-overlapping medium-sized patches (Dimension-wise segment로 발전시킴)
- Swin Transformer – use window to reduce complexity and build hierarchical feature maps

4. METHODOLOGY

MTS(Multi Time-Series) Forecasting



Previous Transformer-based models

- Input $x_{1:T} \rightarrow \{h_1, h_2, \dots, h_T\}$ (Hidden Vector)
- Embed data points at the same time step into a vector
- Capture the dependency among the hidden T vectors
- Capture cross-time dependency, while not explicitly cross-dimension dependency

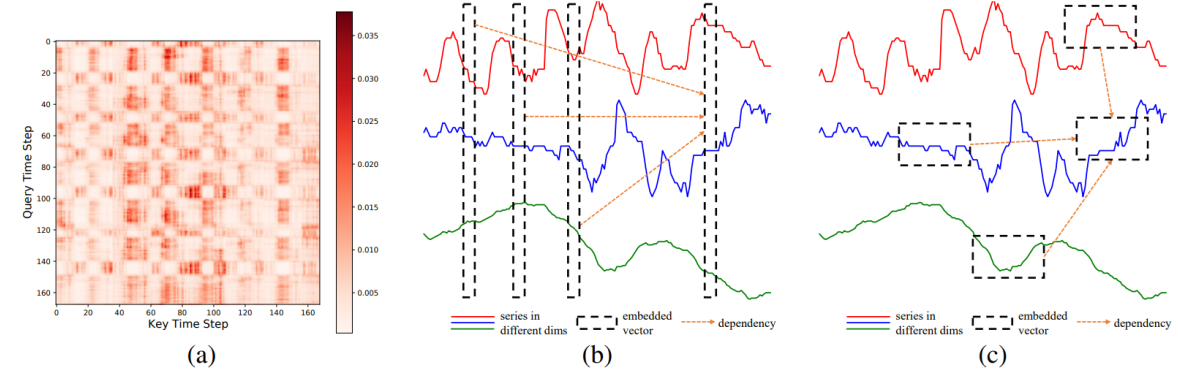


Figure 1: Illustration for our DSW embedding. (a) Self-attention scores from a 2-layer Transformer trained on ETTh1, showing that MTS data tends to be segmented. (b) Embedding method of previous Transformer-based models (Li et al., 2019b; Zhou et al., 2021; Wu et al., 2021a; Liu et al., 2021a): data points in different dimensions at the same step are embedded into a vector. (c) DSW embedding of Crossformer: in each dimension, nearby points over time form a segment for embedding.

4. METHODOLOGY

Dimension-Segment-Wise(DSW)

Object : embedded vector should represent a series segment of single dimension

$$\mathbf{x}_{1:T} = \left\{ \mathbf{x}_{i,d}^{(s)} \mid 1 \leq i \leq \frac{T}{L_{seg}}, 1 \leq d \leq D \right\} \longrightarrow \text{각 차원 } d \text{에 대해 } L_{seg} \text{라는 고정된 길이의 segment로 나눔}$$

$$\mathbf{x}_{i,d}^{(s)} = \left\{ x_{t,d} \mid (i-1) \times L_{seg} < t \leq i \times L_{seg} \right\} \longrightarrow i\text{-th segment in dimension } d \text{ with length } L_{seg}$$

$$h_{i,d} = \textcircled{E} x_{i,d}^{(s)} + E_{i,d}^{(pos)}$$

Learnable projection Matrix

시간적 위치 정보를 반영(Position Embedding) – 기존과 동일

$$\text{Ex) } \mathbf{x} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \mathbf{E} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \mathbf{h} = \mathbf{E} \cdot \mathbf{x} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$$

4. METHODOLOGY

Dimension-Segment-Wise(DSW)

Object : embedded vector should represent a series segment of single dimension

$$H = \{h_{i,d} | 1 \leq i \leq \frac{T}{L_{seg}}, 1 \leq d \leq D\}$$

Example, T(Time step) = 100, $L_{seg} = 10$, D (Dimension) = 5

1. 세그먼트 수 계산(axis-1) : $\frac{T}{L_{seg}} = \frac{100}{10} = 10$
2. 차원의 수(axis-2) : 5 (온도, 습도, 풍속 etc)
3. 임베딩 벡터의 크기 : 선형 투영 모델(d_{model})을 통해 계산됨 (ex $d_{model} = 2$)

Result : T = 100, $L_{seg} = 10$, D = 5, $d_{model} = 2$, $H \in R^{\frac{T}{L_{seg}} \times D \times d_{model}} = R^{(10 \times 5) \times 2}$

- 해당 과정을 통하여 2D vector의 배열로써 표현이 가능함(시간 * 차원) 이러한 2차원 조합의 벡터들로써 표현됨

4. METHODOLOGY

What is ViT (VisionTransformer)?

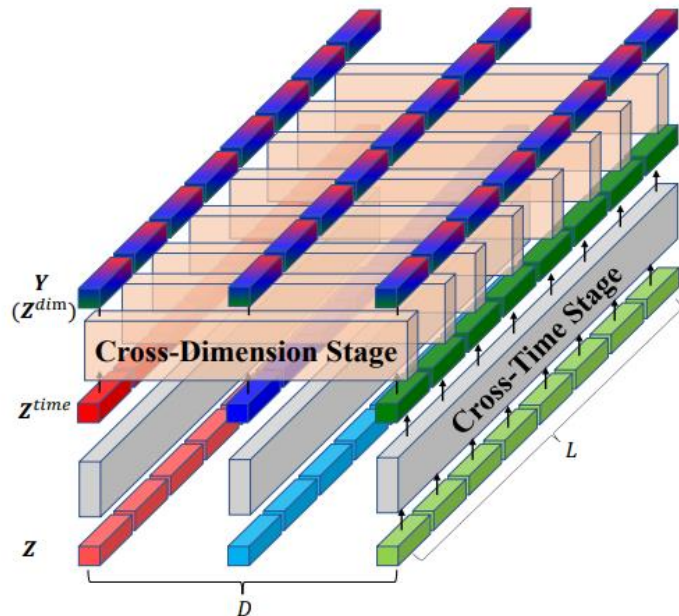


4. METHODOLOGY

Two-Stage Attention(TSA)

ViT Limitation

1. 이미지의 height와 width는 이미지의 길이라는 같은 의미를 가짐 하지만 다변량 시계열에서 time, dimension은 서로 다른 의미를 가지기 때문에 동일한 방식으로 처리 불가
2. 2D array의 self-attention을 바로 적용할 경우, $O(D^2 \frac{T^2}{L_{seg}^2})$ 라는 무지막지한 복잡도!



Flatten 하지 않는
방식 TSA를 고안함

4. METHODOLOGY

Two-Stage Attention(TSA) – Cross-Time stage

Object : 각 dimension 내에서 time step 간의 관계 파악

$$\hat{\mathbf{Z}}_{:,d}^{time} = \text{LayerNorm}\left(\mathbf{Z}_{:,d} + \text{MSA}^{time}(\mathbf{Z}_{:,d}, \mathbf{Z}_{:,d}, \mathbf{Z}_{:,d})\right)$$

MSA : multi-head self-attention

특정 차원 d의 모든 시간 단계에 해당하는 Vector sequence

각 시간 단계가 다른 시간 단계에 어떻게 의존하는지를 학습

$$\mathbf{Z}^{time} = \text{LayerNorm}\left(\hat{\mathbf{Z}}^{time} + \text{MLP}(\hat{\mathbf{Z}}^{time})\right)$$

MLP : Multi-Layer Perceptron(Two layer)

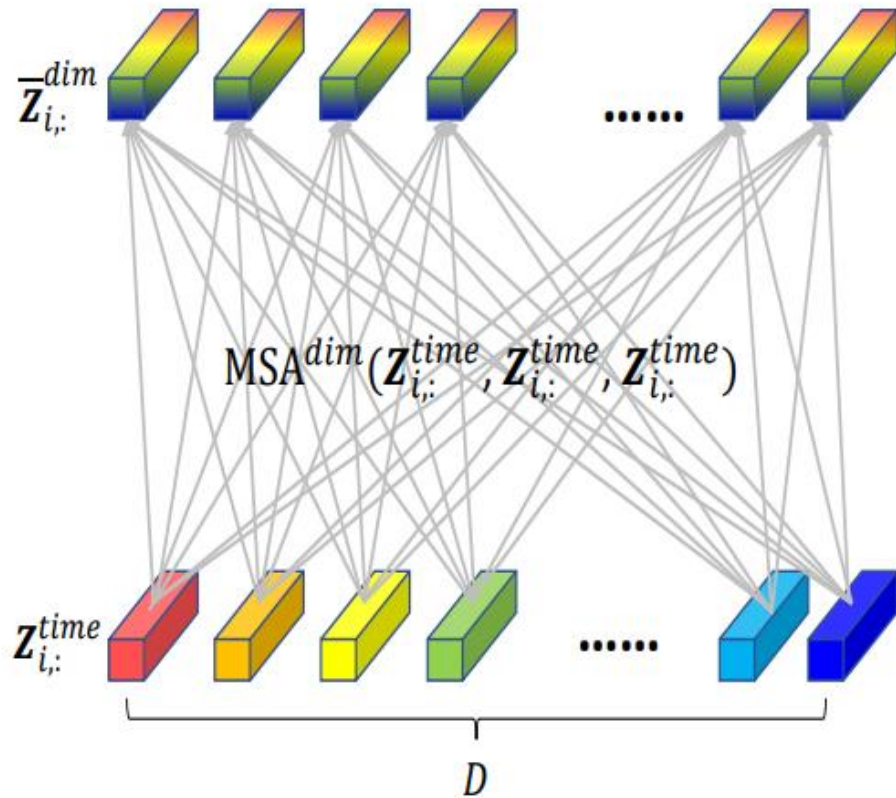
Input of Cross-Dimension Stage

- MSA : 각 차원 안에서 시간 단계 간의 관계 학습(모든 차원에서 동일한 MSA를 사용함 – 계산 효율성)
- MLP : 추가적인 비선형 변환을 적용하여 학습을 용이하게 함

4. METHODOLOGY

Two-Stage Attention(TSA) – Cross-Dimension stage

Object : Capture cross-dimension dependency

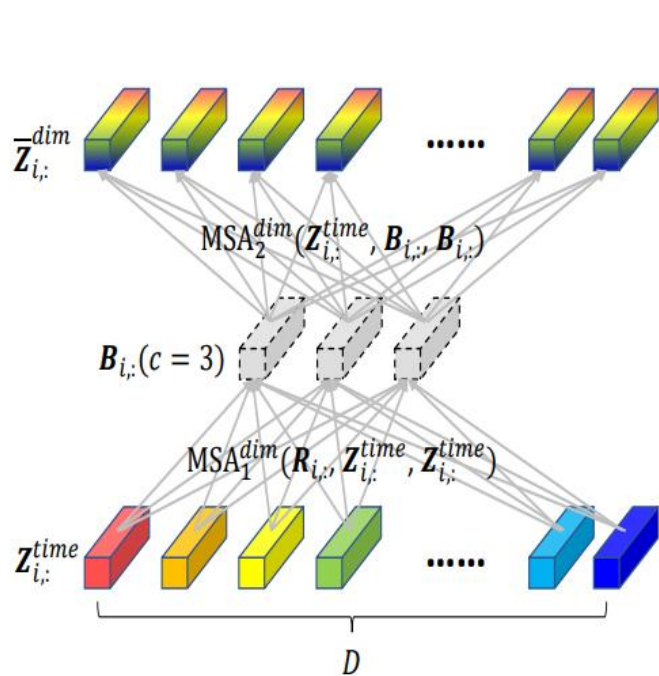


모든 차원 간의 관계를 학습시에 $O(D^2)$
매우 복잡함

4. METHODOLOGY

Two-Stage Attention(TSA) – Cross-Dimension stage

- Router Mechanism(Optimization mechanism)



$$\mathbf{B}_{i,:} = \text{MSA}_1^{dim}(\mathbf{R}_{i,:}, \mathbf{Z}_{i,:}^{time}, \mathbf{Z}_{i,:}^{time}), 1 \leq i \leq L \longrightarrow \text{차원간 상호작용 반영}$$

$$\bar{\mathbf{Z}}_{i,:}^{dim} = \text{MSA}_2^{dim}(\mathbf{Z}_{i,:}^{time}, \mathbf{B}_{i,:}, \mathbf{B}_{i,:}), 1 \leq i \leq L \longrightarrow \text{시간 의존성 + 차원 상호작용}$$

$$\hat{\mathbf{Z}}^{dim} = \text{LayerNorm}(\mathbf{Z}^{time} + \bar{\mathbf{Z}}^{dim})$$

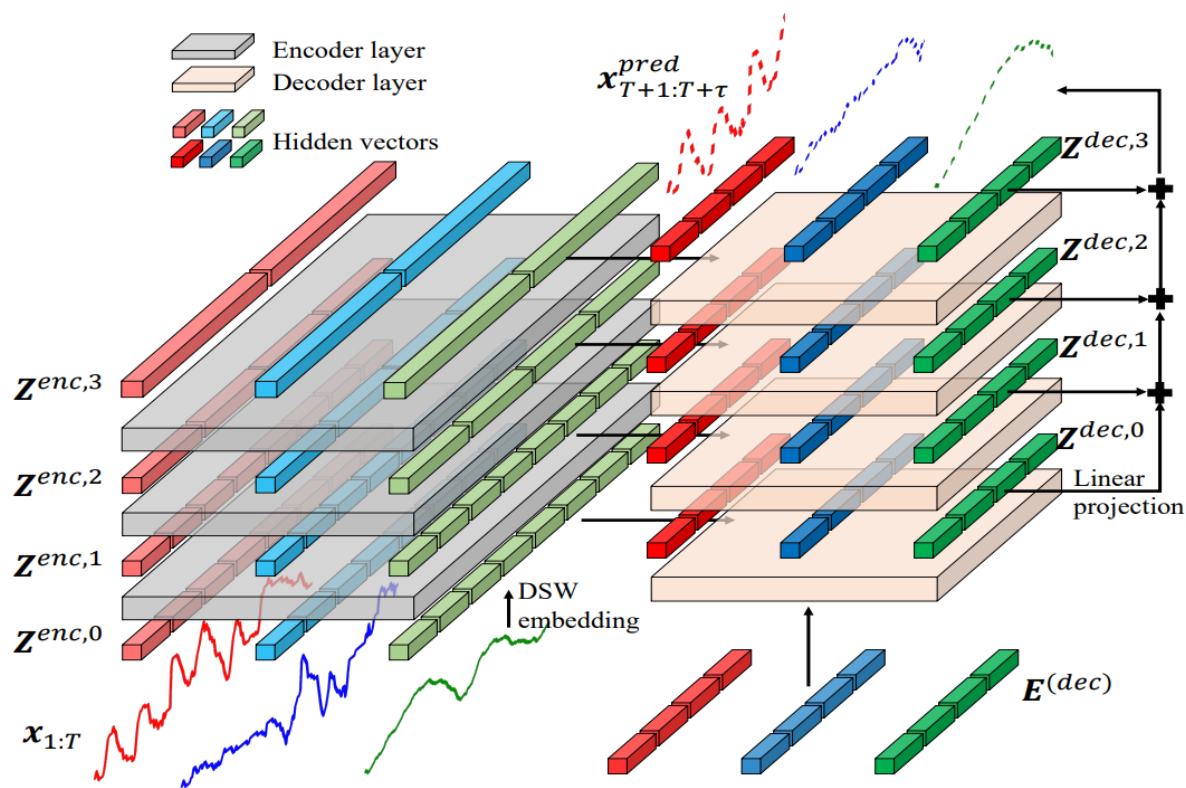
$$\mathbf{Z}^{dim} = \text{LayerNorm}(\hat{\mathbf{Z}}^{dim} + \text{MLP}(\hat{\mathbf{Z}}^{dim}))$$

R_i : Roter Vector
L : Number of segments

- Router : 모든 차원의 정보가 아닌 중요한 정보를 요약 수집, 차원 간의 상호작용 계산(nn.parameter)
- Hierarchical encoder-decoder (HED) : DSW embedding + TSA layer

4. METHODOLOGY

Hierarchical encoder-decoder (HED) : DSW embedding + TSA layer



Encoder : 각 layer의 스케일을 절반씩 줄여가며 계층적으로 시계열 특징을 파악함

Decoder : encoder와는 다르게 각각의 레이어에서 추출된 정보는 학습된 가중치들을 고려하여 계산됨

4. METHODOLOGY

Hierarchical encoder-decoder (HED) : DSW embedding + TSA layer

Encoder

$$\begin{cases} l = 1 : \hat{\mathbf{Z}}^{enc,l} = \mathbf{H} \\ l > 1 : \hat{\mathbf{Z}}_{i,d}^{enc,l} = \mathbf{M}[\mathbf{Z}_{2i-1,d}^{enc,l-1} \cdot \mathbf{Z}_{2i,d}^{enc,l-1}], 1 \leq i \leq \frac{L_{l-1}}{2}, 1 \leq d \leq D \end{cases}$$

concat
↓

$$\mathbf{Z}^{enc,l} = \text{TSA}(\hat{\mathbf{Z}}^{enc,l})$$

$\mathbf{M}$: learnable matrix for segment merging

Decoder

$$\begin{cases} l = 0 : \tilde{\mathbf{Z}}^{dec,l} = \text{TSA}(\mathbf{E}^{(dec)}) \\ l > 0 : \tilde{\mathbf{Z}}^{dec,l} = \text{TSA}(\mathbf{Z}^{dec,l-1}) \end{cases}$$

$$\bar{\mathbf{Z}}_{:,d}^{dec,l} = \text{MSA}(\tilde{\mathbf{Z}}_{:,d}^{dec,l}, \mathbf{Z}_{:,d}^{enc,l}, \mathbf{Z}_{:,d}^{enc,l}), 1 \leq d \leq D$$

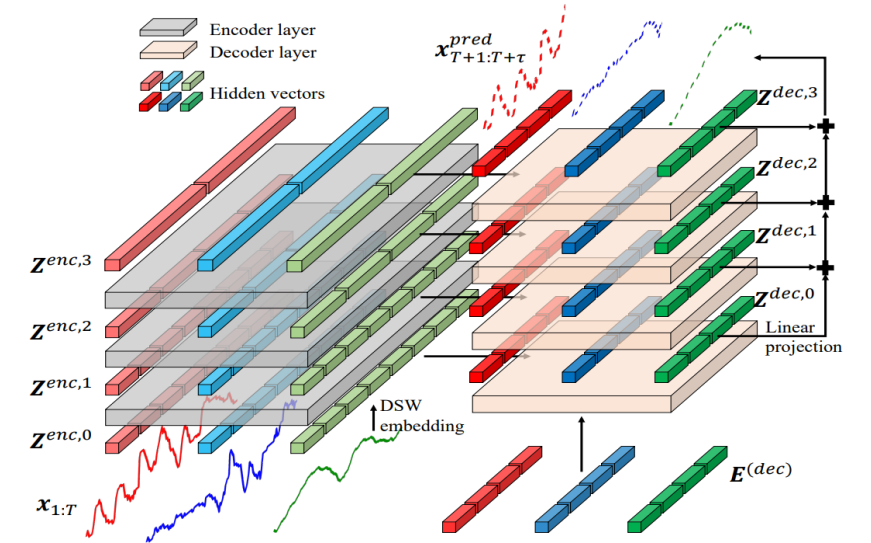
$$\hat{\mathbf{Z}}^{dec,l} = \text{LayerNorm}(\tilde{\mathbf{Z}}^{dec,l} + \bar{\mathbf{Z}}^{dec,l})$$

$$\mathbf{Z}^{dec,l} = \text{LayerNorm}(\hat{\mathbf{Z}}^{dec,l} + \text{MLP}(\hat{\mathbf{Z}}^{dec,l}))$$

Prediction

$$\text{for } l = 0, \dots, N : \mathbf{x}_{i,d}^{(s),l} = \mathbf{W}^l \mathbf{Z}_{i,d}^{dec,l} \quad \mathbf{x}_{T+1:T+\tau}^{pred,l} = \left\{ \mathbf{x}_{i,d}^{(s),l} \mid 1 \leq i \leq \frac{\tau}{L_{seg}}, 1 \leq d \leq D \right\}$$

$$\mathbf{x}_{T+1:T+\tau}^{pred} = \sum_{l=0}^N \mathbf{x}_{T+1:T+\tau}^{pred,l}$$



$\mathbf{W} : \mathbf{Z}_{i,d}^{dec,l}$ 를 최종 예측으로 변환하는 선형 함수(learnable function) – nn.Linear사용함

5. Experiments

Dimension-Segment-Wise(DSW)

Table 1: MSE/MAE with different prediction lengths. Bold/underline indicates the best/second. Results of LSTMa, LSTnet, Transformer, Informer on the first 4 datasets are from [Zhou et al. \(2021\)](#).

Models		LSTMa		LSTnet		MTGNN		Transformer		Informer		Autoformer		Pyraformer		FEDformer		Crossformer	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	24	0.650	0.624	1.293	0.901	0.336	0.393	0.620	0.577	0.577	0.549	0.439	0.440	0.493	0.507	<u>0.318</u>	<u>0.384</u>	0.305	0.367
	48	0.720	0.675	1.456	0.960	0.386	0.429	0.692	0.671	0.685	0.625	0.429	0.442	0.554	0.544	0.342	<u>0.396</u>	<u>0.352</u>	0.394
	168	1.212	0.867	1.997	1.214	0.466	0.474	0.947	0.797	0.931	0.752	0.493	0.479	0.781	0.675	<u>0.412</u>	<u>0.449</u>	0.410	0.441
	336	1.424	0.994	2.655	1.369	0.736	0.643	1.094	0.813	1.128	0.873	0.509	0.492	0.912	0.747	<u>0.456</u>	<u>0.474</u>	0.440	0.461
	720	1.960	1.322	2.143	1.380	0.916	0.750	1.241	0.917	1.215	0.896	0.539	0.537	0.993	0.792	<u>0.521</u>	0.515	0.519	<u>0.524</u>
ETTm1	24	0.621	0.629	1.968	1.170	<u>0.260</u>	<u>0.324</u>	0.306	0.371	0.323	0.369	0.410	0.428	0.310	0.371	0.290	0.364	0.211	0.293
	48	1.392	0.939	1.999	1.215	0.386	0.408	0.465	0.470	0.494	0.503	0.485	0.464	0.465	0.464	<u>0.342</u>	<u>0.396</u>	0.300	0.352
	96	1.339	0.913	2.762	1.542	0.428	0.446	0.681	0.612	0.678	0.614	0.502	0.476	0.520	0.504	<u>0.366</u>	<u>0.412</u>	0.320	0.373
	288	1.740	1.124	1.257	2.076	0.469	0.488	1.162	0.879	1.056	0.786	0.604	0.522	0.729	0.657	0.398	<u>0.433</u>	<u>0.404</u>	0.427
	672	2.736	1.555	1.917	2.941	0.620	0.571	1.231	1.103	1.192	0.926	0.607	0.530	0.980	0.678	0.455	0.464	<u>0.569</u>	<u>0.528</u>
WTH	24	0.546	0.570	0.615	0.545	0.307	<u>0.356</u>	0.349	0.397	0.335	0.381	0.363	0.396	<u>0.301</u>	0.359	0.357	0.412	0.294	0.343
	48	0.829	0.677	0.660	0.589	0.388	0.422	0.386	0.433	0.395	0.459	0.456	0.462	<u>0.376</u>	<u>0.421</u>	0.428	0.458	0.370	0.411
	168	1.038	0.835	0.748	0.647	<u>0.498</u>	<u>0.512</u>	0.613	0.582	0.608	0.567	0.574	0.548	0.519	0.521	0.564	0.541	0.473	0.494
	336	1.657	1.059	0.782	0.683	<u>0.506</u>	<u>0.523</u>	0.707	0.634	0.702	0.620	0.600	0.571	0.539	0.543	0.533	0.536	0.495	0.515
	720	1.536	1.109	0.851	0.757	0.510	0.527	0.834	0.741	0.831	0.731	0.587	0.570	0.547	0.553	0.562	0.557	<u>0.526</u>	<u>0.542</u>
ECL	48	0.486	0.572	0.369	0.445	<u>0.173</u>	<u>0.280</u>	0.334	0.399	0.344	0.393	0.241	0.351	0.478	0.471	0.229	0.338	0.156	0.255
	168	0.574	0.602	0.394	0.476	<u>0.236</u>	<u>0.320</u>	0.353	0.420	0.368	0.424	0.299	0.387	0.452	0.455	0.263	0.361	0.231	0.309
	336	0.886	0.795	0.419	0.477	0.328	<u>0.373</u>	0.381	0.439	0.381	0.431	0.375	0.428	0.463	0.456	0.305	0.386	<u>0.323</u>	0.369
	720	1.676	1.095	0.556	0.565	0.422	0.410	0.391	0.438	0.406	0.443	<u>0.377</u>	0.434	0.480	0.461	0.372	0.434	0.404	<u>0.423</u>
	960	1.591	1.128	0.605	0.599	0.471	0.451	0.492	0.550	0.460	0.548	0.366	0.426	0.550	0.489	<u>0.393</u>	0.449	0.433	<u>0.438</u>
ILI	24	4.220	1.335	4.975	1.660	4.265	1.387	3.954	1.323	4.588	1.462	3.101	1.238	3.970	1.338	2.687	1.147	<u>3.041</u>	<u>1.186</u>
	36	4.771	1.427	5.322	1.659	4.777	1.496	4.167	1.360	4.845	1.496	<u>3.397</u>	1.270	4.377	1.410	2.887	1.160	3.406	<u>1.232</u>
	48	4.945	1.462	5.425	1.632	5.333	1.592	4.746	1.463	4.865	1.516	<u>2.947</u>	<u>1.203</u>	4.811	1.503	2.797	1.155	3.459	1.221
	60	5.176	1.504	5.477	1.675	5.070	1.552	5.219	1.553	5.212	1.576	<u>3.019</u>	<u>1.202</u>	5.204	1.588	2.809	1.163	3.640	1.305
Traffic	24	0.668	0.378	0.648	0.403	<u>0.506</u>	<u>0.278</u>	0.597	0.332	0.608	0.334	0.550	0.363	0.606	0.338	0.562	0.375	0.491	0.274
	48	0.709	0.400	0.709	0.425	0.512	<u>0.298</u>	0.658	0.369	0.644	0.359	0.595	0.376	0.619	0.346	0.567	0.374	<u>0.519</u>	0.295
	168	0.900	0.523	0.713	0.435	<u>0.521</u>	<u>0.319</u>	0.664	0.363	0.660	0.391	0.649	0.407	0.635	0.347	0.607	0.385	0.513	0.289
	336	1.067	0.599	0.741	0.451	<u>0.540</u>	<u>0.335</u>	0.654	0.358	0.747	0.405	0.624	0.388	0.641	0.347	0.624	0.389	0.530	0.300
	720	1.461	0.787	0.768	0.474	0.557	<u>0.343</u>	0.685	0.370	0.792	0.430	0.674	0.417	0.670	0.364	0.623	0.378	<u>0.573</u>	0.313

Model : LSTMa, LSTnet, MTGNN, Transformer, Informer, Pyraformer, FEDformer, Crossformer

Dataset

- ETTh1, ETTh2 : Electricity Transformer Temperature-hourly, minutely
- WTH : weather
- ECL : Electricity Consuming Load
- ILI : Influenza-Like Illness
- Traffic : Road occupancy rates

36 top-1 and 51 top-2

5. Experiments

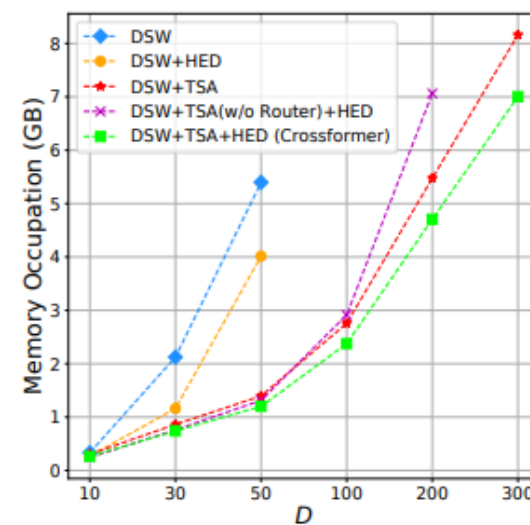
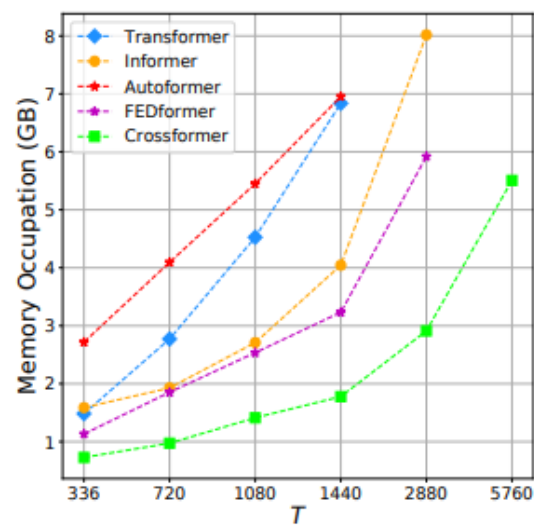
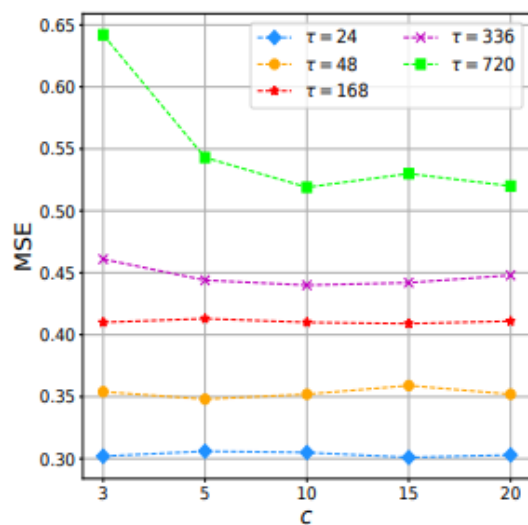
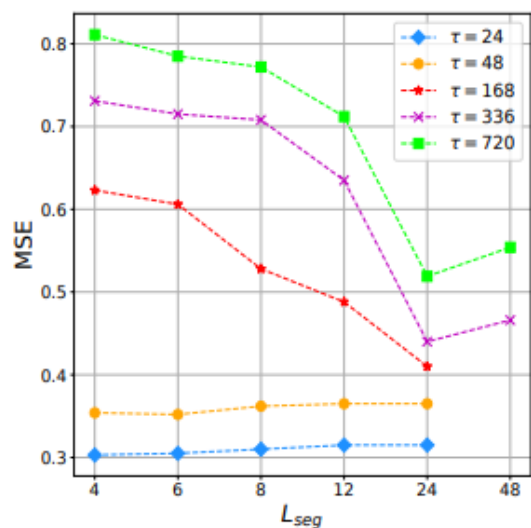
Ablation study(Crossformer Component)

Table 2: Component ablation of Crossformer: DSW embedding, TSA layer and HED on ETTh1.

Models	Transformer		DSW		DSW+TSA		DSW+HED		DSW+TSA+HED	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
24	0.620	0.577	0.373	0.418	0.322	0.373	0.406	0.454	0.305	0.367
48	0.692	0.671	0.456	0.479	0.365	0.403	0.493	0.512	0.352	0.394
168	0.947	0.797	0.947	0.731	0.473	0.479	0.614	0.583	0.410	0.441
336	1.094	0.813	0.969	0.752	0.553	0.534	0.788	0.676	0.440	0.461
720	1.241	0.971	1.086	0.814	0.636	0.599	0.841	0.717	0.519	0.524

5. Experiments

Evaluation on hyper-parameter and computational efficiency



6. Conclusion

Conclusion

- DSW(Dimension-Segment-Wise) Embedding 방법을 사용하여 time, dimension의 정보를 최대한 보존함
- TSA를 사용하여 cross-time, cross-dimension dependency를 capture함
- HED를 사용하여 Different scale에서의 정보를 사용 더 높은 성능의 예측 수행
- 6개의 데이터셋에서 SOTA 모델 달성

Limitation

- Cross-Dimension의 full connection은 고차원 데이터의 noise를 학습할 수 있음
- DLinear(Appendix B.2)를 보면 성능이 매우 좋은 것을 확인 가능(단 변수간 의존성은 고려 x)

Opinion

- 저자도 얘기하지만 대용량 데이터셋이 현재 NLP, CV에 비해 정형화되거나 체계적이지 않음
- Chatgpt처럼 거대 시계열 데이터로 학습한다면 모든 도메인에 어느 정도 정확성을 보장하는 모델이 나오지 않을까??

Model	Metric	MTDNN		Transformer		Infotree		Autoformer		Pyrformer		FEDformer		STformer		Crossformer		DLinear	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE		
ETTh1	24	0.136	0.399	0.620	0.577	0.577	0.549	0.429	0.440	0.493	0.507	0.318	0.364	0.368	0.441	0.300	0.307	0.322	0.305
	48	0.106	0.429	0.692	0.671	0.680	0.625	0.429	0.442	0.554	0.544	0.342	0.396	0.445	0.465	0.352	0.379	0.352	0.283
	168	0.066	0.474	0.947	0.797	0.901	0.752	0.449	0.479	0.781	0.675	0.511	0.449	0.652	0.608	0.416	0.441	0.416	0.430
	336	0.036	0.643	1.094	0.813	1.128	0.673	0.599	0.492	0.912	0.747	0.456	0.474	1.009	0.806	0.440	0.451	0.450	0.482
	720	0.016	0.750	1.241	0.867	1.215	0.696	0.559	0.537	0.993	0.792	0.521	0.523	1.071	0.817	0.515	0.524	0.406	0.401
ETTh2	24	0.200	0.320	0.306	0.371	0.322	0.369	0.410	0.428	0.310	0.371	0.290	0.364	0.278	0.348	0.311	0.291	0.212	0.289
	48	0.136	0.408	0.465	0.470	0.494	0.503	0.485	0.464	0.465	0.464	0.342	0.396	0.445	0.458	0.389	0.352	0.278	0.230
	168	0.028	0.440	0.603	0.612	0.678	0.614	0.582	0.476	0.520	0.560	0.366	0.412	0.620	0.655	0.320	0.371	0.318	0.264
	336	0.009	0.488	1.182	0.879	1.058	0.786	0.604	0.522	0.729	0.657	0.398	0.431	0.733	0.597	0.401	0.427	0.369	0.266
	720	0.020	0.571	1.251	1.103	1.192	0.926	0.607	0.580	0.980	0.678	0.423	0.464	0.777	0.625	0.369	0.378	0.414	0.417
WTH1	24	0.307	0.320	0.349	0.397	0.335	0.381	0.363	0.396	0.301	0.350	0.357	0.412	0.307	0.359	0.294	0.343	0.357	0.391
	48	0.188	0.422	0.388	0.411	0.395	0.429	0.456	0.462	0.328	0.422	0.428	0.458	0.381	0.416	0.279	0.411	0.425	0.444
	168	0.098	0.521	0.613	0.582	0.608	0.567	0.574	0.548	0.519	0.521	0.564	0.541	0.657	0.582	0.477	0.494	0.525	0.516
	336	0.056	0.521	0.707	0.654	0.702	0.620	0.660	0.571	0.539	0.543	0.533	0.556	0.566	0.564	0.495	0.515	0.556	0.537
	720	0.010	0.627	0.834	0.761	0.831	0.711	0.867	0.570	0.547	0.553	0.562	0.597	0.580	0.582	0.520	0.542	0.562	0.571
ETC1	48	0.173	0.280	0.334	0.399	0.344	0.393	0.241	0.351	0.478	0.471	0.229	0.338	0.356	0.432	0.350	0.258	0.455	0.258
	168	0.026	0.320	0.353	0.420	0.368	0.424	0.299	0.387	0.425	0.451	0.283	0.361	0.516	0.527	0.321	0.309	0.195	0.205
	336	0.028	0.372	0.381	0.419	0.381	0.431	0.375	0.428	0.463	0.456	0.395	0.386	OOM	OOM	0.321	0.309	0.238	0.216
	720	0.022	0.420	0.391	0.438	0.406	0.443	0.377	0.434	0.480	0.461	0.371	0.434	OOM	OOM	0.404	0.415	0.272	0.266
	960	0.071	0.451	0.492	0.550	0.460	0.548	0.366	0.426	0.550	0.489	0.393	0.449	OOM	OOM	0.453	0.438	0.299	0.267
EE	24	4.365	1.387	3.954	3.323	4.588	1.462	1.101	1.238	3.970	1.338	2.687	1.147	3.150	1.233	3.041	1.161	2.940	2.205
	36	4.777	1.486	4.167	3.860	4.845	1.496	3.267	1.270	4.377	1.410	2.807	1.160	3.512	1.243	3.806	1.232	2.826	1.181
	48	5.333	1.592	4.746	4.463	4.865	1.516	2.947	1.203	4.811	1.503	2.725	1.185	3.499	1.234	3.459	1.231	2.677	1.155
	60	5.070	1.552	4.229	3.553	5.212	1.576	3.019	1.202	5.264	1.586	2.809	1.163	3.715	1.316	3.640	1.305	3.011	1.245
	720	0.557	0.343	0.685	0.370	0.792	0.430	0.674	0.417	0.670	0.364	0.623	0.378	OOM	OOM	0.377	0.313	0.453	0.213
Traffic	48	0.012	0.290	0.450	0.389	0.644	0.359	0.590	0.378	0.619	0.348	0.567	0.374	OOM	OOM	0.519	0.225	0.270	0.276
	168	0.021	0.310	0.664	0.363	0.660	0.291	0.649	0.407	0.635	0.347	0.607	0.365	OOM	OOM	0.513	0.289	0.395	0.277
	336	0.040	0.310	0.654	0.358	0.747	0.405	0.624	0.368	0.641	0.347	0.624	0.369	OOM	OOM	0.520	0.300	0.415	0.269
	720	0.027	0.343	0.685	0.370	0.792	0.430	0.674	0.417	0.670	0.364	0.623	0.378	OOM	OOM	0.513	0.313	0.453	0.213

THANK YOU
